

SOFTWARE DESIGN 2026 / 02

Pythonの基礎

小さな処理を、説明できるコードに

情報システム学科 3年生 選択 2単位

今日の到達目標

- 値・変数・条件分岐・反復を使って処理を書く。
- リストと辞書の役割を説明する。
- 関数の入力と出力を決め、例で確かめる。
- エラーの原因を読み取り、小さく修正する。

完成物：架空の注文データから合計を計算するプログラム。

ガイドンスからの接続

第1回で確認した開発環境を使う。Pythonは **3.12以上**。

```
python3 --version  
python3 examples/02-basics/order_summary.py
```

エディタでファイルを保存し、ターミナルで実行する。
実行前に出力を予測し、実行後に予測との差を説明する。

今日扱う注文データ

商品	単価（円）	個数	小計（円）
ノート	200	2	400
ペン	120	3	360
合計			760

データは教材用に作ったもの。入力と期待値を先に固定する。

値と型

```
price = 200          # int: 整数
ratio = 0.1          # float: 浮動小数点数
name = "ノート"      # str: 文字列
available = True      # bool: 真偽値
```

同じ `+` でも、整数なら加算、文字列なら連結になる。

```
print(200 + 120)      # 320
print("200" + "120")  # 200120
```

代入と比較

```
quantity = 2
quantity = quantity + 1
print(quantity)      # 3
print(quantity == 3) # True
```

`=` は名前と値を結び付ける。`==` は値が等しいか調べる。

予測： `quantity == 2` を実行しても、値は変わるだろうか。

演算子と括弧

```
print(2 + 3 * 4)      # 14
print((2 + 3) * 4)    # 20
print(7 // 2)         # 3
print(7 % 2)          # 1
```

括弧で計算の意図を示す。 `/` と `//` は役割が異なる。

金額の例は円単位の整数とし、端数処理を持ち込まない。

条件分岐と境界

```
quantity = 0
if quantity < 0:
    print("個数は0以上にしてください")
elif quantity == 0:
    print("注文なし")
else:
    print("注文あり")
```

境界の `-1`、`0`、`1` を試す。インデントが処理の範囲を決める。

リストは順序のある集まり

```
prices = [200, 120, 80]
print(prices[0])      # 200
print(len(prices))    # 3
prices.append(150)
```

添字は0から始まる。存在しない添字は `IndexError`。

`[]` も有効なリスト。空の場合の扱いを考える。

反復で合計を作る

```
prices = [200, 120, 80]
total = 0
for price in prices:
    total = total + price
print(total) # 400
```

反復後	1回目	2回目	3回目
total	200	320	400

rangeと反復回数

```
for index in range(3):  
    print(index)  
# 0, 1, 2 を順に出力
```

`range(3)` の終端3は含まない。

実験： `range(0)`、`range(1, 4)` の出力を予測して試す。

辞書はキーで値を取り出す

```
order = {"name": "ノート", "price": 200,  
         "quantity": 2}  
subtotal = order["price"] * order["quantity"]  
print(subtotal)  # 400
```

添字の位置でなく、意味のあるキーで項目を表す。

存在しないキーの参照は `KeyError` になる。

複数の注文を表す

```
orders = [  
    {"name": "ノート", "price": 200, "quantity": 2},  
    {"name": "ペン", "price": 120, "quantity": 3},  
]  
total = 0  
for order in orders:  
    total += order["price"] * order["quantity"]  
print(total)  # 760
```

リストが注文の集まり、辞書が1件の注文を表す。

関数の入力と出力

```
def subtotal(price: int, quantity: int) -> int:  
    return price * quantity
```

```
amount = subtotal(200, 2)
```

```
print(amount)  # 400
```

`return` で結果を呼び出し元に返す。型注釈は意図を伝える。

型注釈だけでは、実行時の入力検証は行わない。

printとreturnの違い

操作	結果の行き先	次の計算に使う
<code>print(400)</code>	画面へ表示	戻り値は <code>None</code>
<code>return 400</code>	呼び出し元へ	数値400を使える

表示と計算を分けると、画面がなくても関数をテストできる。

仕様を例にする

```
assert subtotal(200, 2) == 400
assert subtotal(200, 0) == 0
assert subtotal(0, 3) == 0
```

通常の例と境界の例を用意する。失敗した例が修正の手掛かりになる。

`assert` は学習・テスト用。入力検証には明示的な条件分岐を使う。

入力の契約

この教材の注文は、単価と個数が **0以上の整数**。

```
def checked_subtotal(price, quantity):  
    if (type(price) is not int  
        or type(quantity) is not int):  
        raise ValueError("単価と個数は整数")  
    if price < 0 or quantity < 0:  
        raise ValueError("単価と個数は0以上")  
    return price * quantity
```

Pythonの `bool` は `int` の派生型。ここでは真偽値を受け付けない。

エラーの読み取り

```
Traceback (most recent call last):  
  File "demo.py", line 2, in <module>  
    print(int("two"))  
ValueError: invalid literal for int() ...
```

最後の行で種類と内容を確認し、その上で発生位置を読む。
最小の入力で再現してから、修正を試す。

例外と回復

```
text = "two"
try:
    quantity = int(text)
except ValueError:
    print("整数を入力してください")
else:
    print(quantity)
```

回復できる失敗を扱う。原因を隠すために全例外を握りつぶさない。

変更が共有されるリスト

```
a = [200, 120]
b = a
b.append(80)
print(a) # [200, 120, 80]
c = a.copy()
c.append(150)
```

`a` と `b` は同じリストを参照する。 `copy()` は浅いコピー。
入れ子のリストや辞書まで複製するとは限らない。

実演の観察ポイント

```
python3 examples/02-basics/order_summary.py  
python3 -m unittest discover -s tests -p test_foundations.py -v
```

1. 合計が760円になることを確認する。
2. 計算関数と、読み込み・表示の処理の境界を探す。
3. 不正なデータをどこで拒否するか説明する。

演習A 予測と追跡

前の「反復で合計を作る」を手入力して、各反復の `total` を記録する。

- 入力を空リストに変更する。
- 同じ価格を2回含める。
- リストの順番を変える。

記録：実行前の予測、実際の出力、違いが生じた理由。

演習B 注文集計

`exercises/02.md` の仕様に従い、`submission/02/summary.py` を作る。

入力	期待する結果
ノート2個、ペン3個	760円
注文なし	0円
個数が負	理由の分かるエラー

まず計算関数を作り、その後に表示を加える。

演習C テストを追加する

通常・境界・不正入力から、少なくとも1例ずつ選ぶ。

- その入力を選んだ理由を書く。
- 期待値を手で計算する。
- 修正前に失敗し、修正後に通る例を1つ残す。

AIを使った場合は、採用した提案と検証方法を記録する。

確認問題

1. `range(3)` は何回反復するか。
2. `print()` だけの関数を計算に使うと、何が起こるか。
3. 空の注文の合計を0にする理由は何か。
4. 型注釈があっても、不正入力のテストが必要な理由は何か。

答えをコード例か実行結果で説明する。

今日のまとめと次回

入力の形と結果を決めると、コードを小さく分けられる。
テストは、意図した動作を具体的な例として残す。

復習：演習Bの関数を、画面を見ずに説明する。

次回：AIと学習・開発を進め、提案を検証する。

参考と教材ファイル

- Python 3.12 チュートリアル

<https://docs.python.org/3.12/tutorial/>

- Python 3.12 組み込み型

<https://docs.python.org/3.12/library/stdtypes.html>

- 学生向け課題： `exercises/02.md`

- 実行例： `examples/02-basics/`

例と図は本教材のために作成したもの。